

- 1、直连路由 --- 路由器默认生成的接口直连网段的路由条目
条件：1. 接口双UP；2. 接口必须配置IP地址
- 2、静态路由 --- 管理员手工配置
- 3、动态路由 --- 通过运行某种算法从其他路由器学习

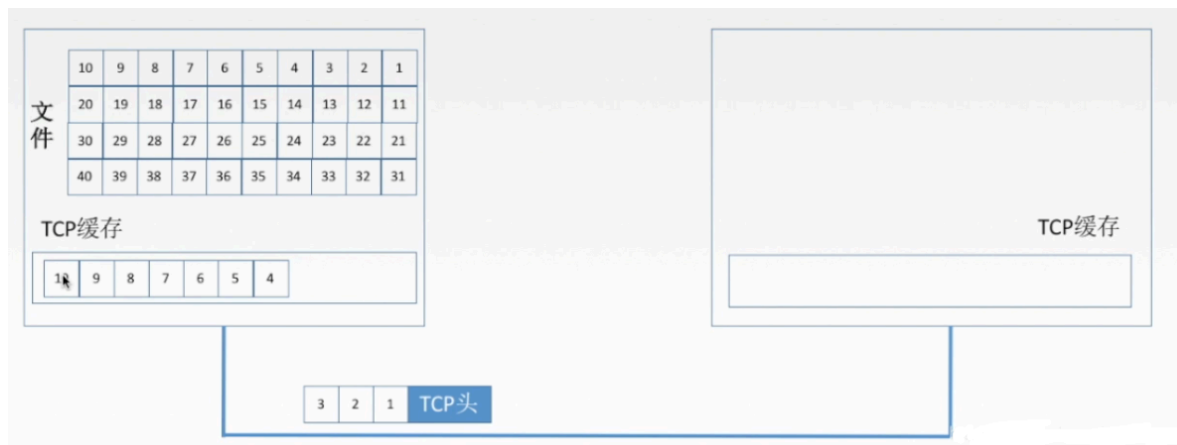
第二章，TCP --- 传输控制协议

是一种面向连接的可靠传输协议

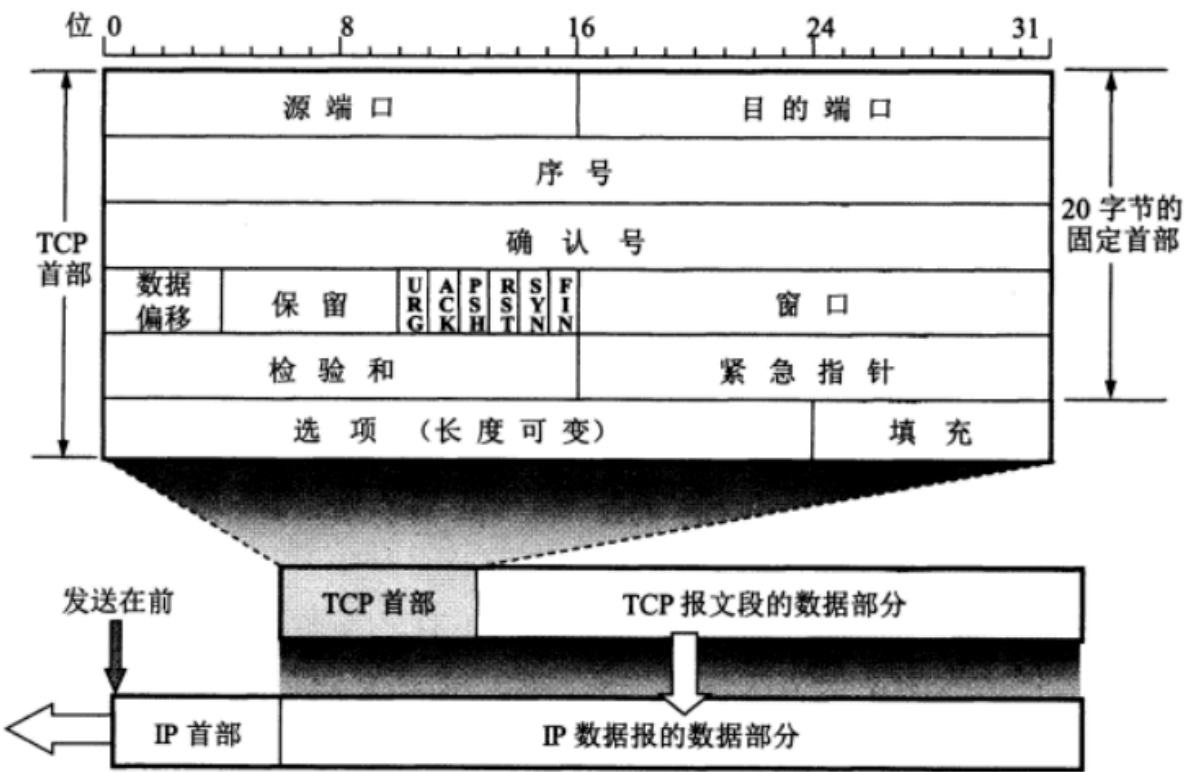
TCP是在不可靠的IP层之上，实现的可靠数据传输协议，**主要解决传输的可靠、有序、无丢失和不重复问题。**

特点：

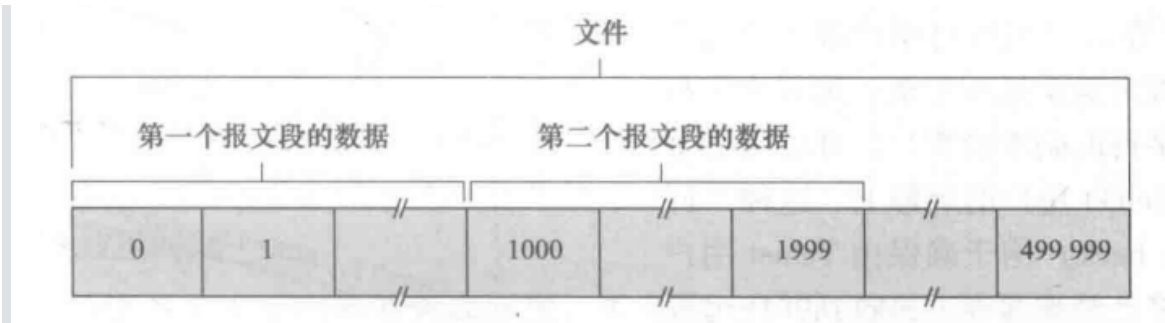
- 存在虚连接
- TCP是点到点连接
- 提供可靠交付的服务
- **TCP提供全双工通讯，且通信双方均具备发送缓存和接收缓存**
 - **发送缓存**
 - TCP准备发送的数据
 - TCP已经发送，但是没有收到确认的数据
 - **接收缓存**
 - 按序到达，但尚未被应用程序读取的数据
 - 未按序到达的数据
- **TCP是面向字节流的协议** --- TCP把应用层交付的数据看做是一串无结构的字节流。



TCP报文段



序列号：是建立在TCP字节流之上的参数，TCP每发送一个字节数据，则序列号加1。



确认序列号：为了实现确认机制诞生的。

- 1. 表达了接收方期望收到发送方发送的下一个字节的序列号。
- 2. 代表接收方已经收到了该确认序列号之前的所有数据。

窗口：指的是，发送本报文段的一方的接收窗口(初始时为接收缓存)大小；即允许发送方连续发送的数据量。

控制位(标志位)：

- **紧急位URG** --- 代表后续数据中，存在紧急数据，是高优先级的数据，**应尽快处理，不要在接收缓存中排队。**
- **确认位ACK** --- ACK=1，则代表**确认序列号有效**；一般而言，数据传输阶段，该标记位均为1
- **推送位PSH** --- 接收方应尽快交付存在于接收缓存中的数据，不需要等待缓存填满再向上交付。
- **复位RST** --- 当TCP连接存在问题，必须释放连接，然后重新建立连接时，使用该标记位。
- **同步位SYN** --- 表示一个连接请求报文
- **终止位FIN** --- 表示一个连接释放报文

TCP的可靠性

确认机制

TCP保证对方能够收到本端发送的数据段的方式，就是让对方回复一个确认报文段；而这个确认报文段最重要的标志就是TCP头部中的ACK=1，同时激活确认序列号。 --- 逐段确认。

因为逐段确认机制，会导致TCP传输效率降低，以及存在窗口值参数，故TCP确认机制变更为**累积确认**。发送多次报文，只需要接收到一次确认即可。

SACK --- 选择确认机制 --- 在确认报文的TCP首部中的Option字段中，携带自己已经收到的数据信息；此时发送方就可以有选择性的重传数据，而不会重传丢失数据及之后的所有数据。**是需要通信双方进行协商的，如果双方都支持，则可以使用。** --- 在TCP三次握手的前两次SYN报文中协商，与Option字段中携带。

延迟确认机制：

- 在没有收到2*最大段长度的数据为止，不做确认。
- 最大延迟0.5秒回复确认报文。

捎带应答 --- 本地发送回执信息时，顺便携带某些数据信息。

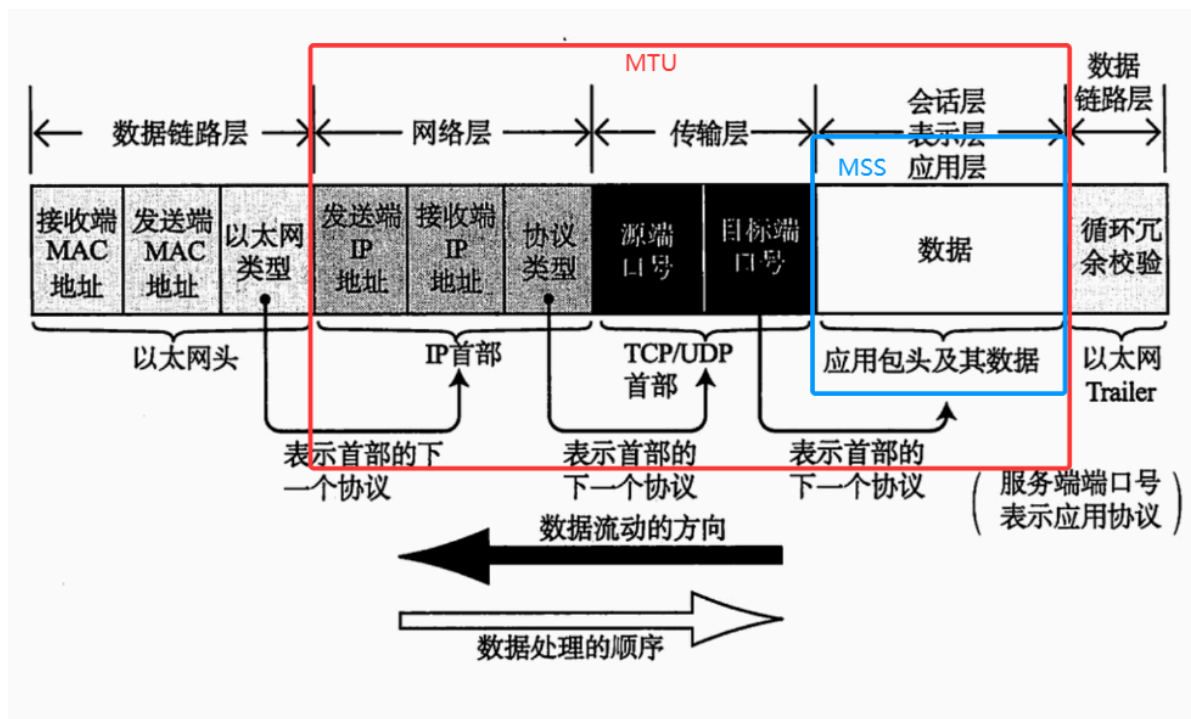
排序机制

排序机制依靠每个分段TCP首部中的序列号字段进行。

MSS(最大段长度) --- 决定数据段会被分割成多大的大小。

该参数是在TCP建立握手过程中，通过前两个SYN报文段来进行协商确定的，被携带在TCP首部的可选项字段中。 --- 如果MSS数值不同，则选择最小的MSS数值作为后续分段的标准。

MSS数值是跟随MTU数值确定的。



分片 --- 是网络层IP协议的概念

如果，在传输层使用了TCP协议，则存在TCP分段，此时，IP层面封装时，不再需要进行进一步的分片。

PMTU --- Path MTU发现机制

重传机制

本地多久没有收到对端的确认报文，则认为此时需要进行重传操作 --- **超时重传**。

RTT --- 往返时间 --- 指的是发送端发送数据后，到本地接收到对端反馈的确认报文的这段时间。

RTO --- 超时重传时间；该时间是略大于RTT时间。

RTT时间是动态变化的，所以RTO时间也是动态变化的。

超时间隔加倍机制 --- 每发生一次超时重传，则RTO时间加倍，直到收到确认报文，RTO时间重新根据RTT时间进行计算。

快速重传机制

正常情况下，服务器收到一个乱序报文，不会意识到自己的报文段丢失，此时，**服务器是不能直接确认后续的报文段的**。采用了**冗余ACK机制** --- 就是将已经发送过的ACK报文，再一次发送。

如果接收方，连续收到三次冗余ACK报文，则认为存在报文丢失现象，根据ACK报文内容，进行数据信息重传。

注：超时重传是发送方自行判断的结果；而快速重传是发送方根据接收方的反馈信息来判断的结果。 --- 这两种重传机制，共同使用。

流控机制

TCP的流量控制，是属于TCP的传输效率问题；

滑动窗口

思路：趁着对方在忙其他任务时，自己多发一些报文信息。

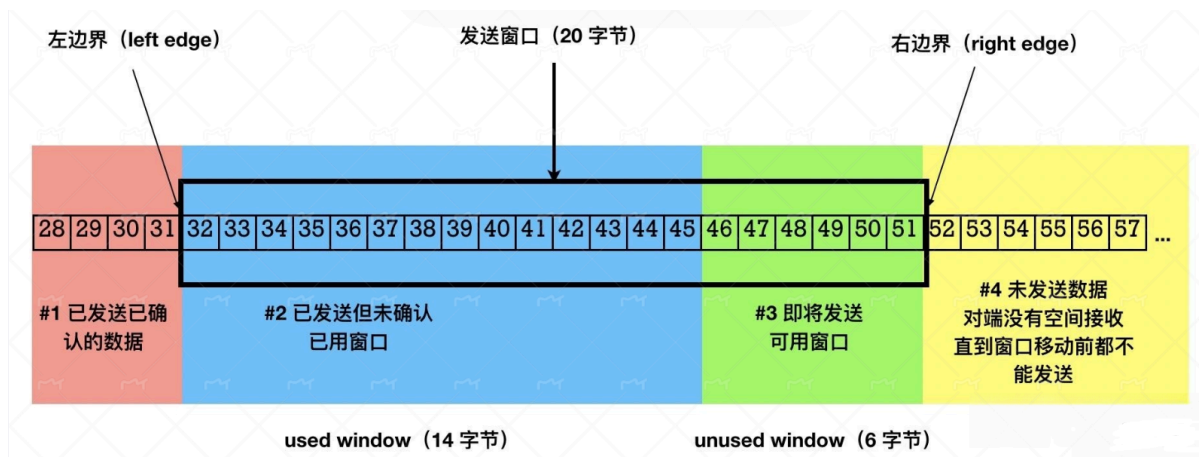
专门引入 --- 窗口值变量

窗口值 --- 大小可变的数值；含义是：无需等待确认应答，而可以连续发送的数据最大量。

窗口的体现，实际上就是缓存区的大小。

- 接收缓存，是由设备的操作系统和当前运行环境决定。
- 发送缓存，最初是由对端发送来的数据报文中的Windows数值决定。

为什么窗口值可以控制流量传输速率 --- 因为窗口值可变，并非是固定的参数。TCP要求发送方根据对方的接收窗口来控制数据的发送量。



发送窗口 = 本地发送缓存 = 对端接收缓存。

窗口关闭

因为窗口值是一个动态变化的数据，即接收端根据自己接收缓存空间的剩余量来确定的数值，该数值是存在归0的可能性。即代表接收方没有能力存储更多的未处理数据。

所以，当剩余接收缓存空间=0时，则会向对方发送一个windows = 0的报文，告知对方，不要在发送数据了，本地无法处理。

当接收方发送一个win = 0的报文，此时，发送方会停止数据发送；等待接收方处理完之前的数据，接收方会发送一个win非0的ACK报文，用来通知发送方继续发送数据。 --- 如果该非0报文丢失，则双方进入死锁状态。

原因：

- 对于发送方而言，需要等待对方发送一个win非0的报文，才可以发送自己的信息
- 对于接收方而言，已经通知对方继续发送，本地只需等待即可

解决方案：让发送方定时发送一个只包含一个字节的探测报文。

在Option字段中，存在一个窗口扩大因子选项；取值是0-14；**将TCP的窗口值左移的位数**。 --- 该参数，只能出现在SYN报文中。

TCP的拥塞控制

流量控制是为了让接收方能够来得及接收，而拥塞控制是为了降低整个网络的拥塞程度。

拥塞 ---> 网络中对资源需求超过了资源可用量的情况。

流控机制主要是为了防止发送方发送过大的流量，导致接收方的接收缓存溢出的问题；拥塞，主要是收到网络环境影响，降低本地的数据吞吐量的现象。

- TCP是如何判断此时网络中发生拥塞
 - **TCP将连接中出现丢包行为，视为拥塞的表现。**
 - 数据包确认超时
 - 收到对方发送的三次冗余ACK
- TCP是如何控制数据的发送量
 - **拥塞窗口** --- 可以对发送的字节数大小进行限制。
 - **发送方发送的数据量(发送窗口 --- 发送缓存) = 接收窗口(Windows)和拥塞窗口的最小值。**
 - **拥塞窗口大小** --- TCP采用自计时行为，通过确认报文来增大拥塞窗口。

○ 拥塞控制算法 --- 慢启动，拥塞避免，快速恢复。

■ 慢启动 --- 拥塞窗口最初会被设定为一个MSS数值大小，即一次发送一个数据报文段。

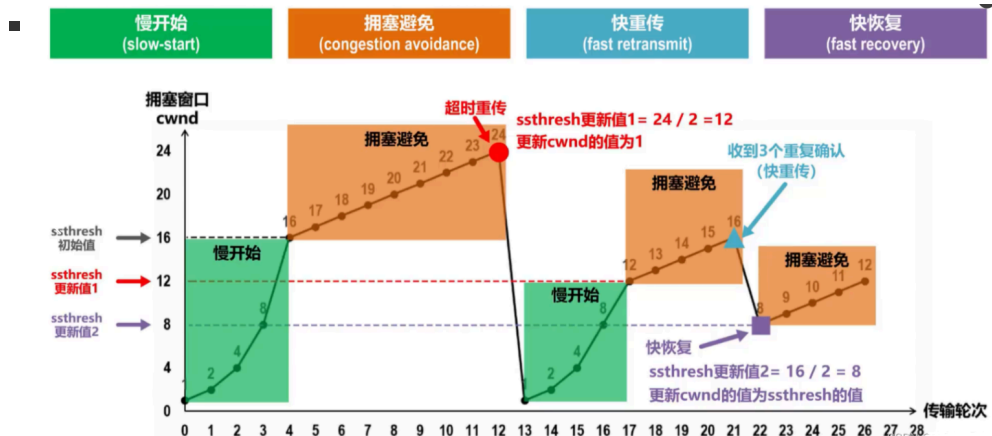
■ 每收到一个新的ACK报文(不包括重传)，就会增大一个MSS的大小。

■ 为了防止拥塞窗口增长过大，造成网络拥塞，我们不能一直使用慢启动算法，故设定了一个慢启动门限；当超出这个门限时，不再使用慢启动算法。

■ 拥塞窗口 < 门限值 --- 使用慢启动算法
拥塞窗口 > 门限值 --- 使用拥塞避免算法

■ 拥塞避免算法 --- 在一个RTT时间内，传输的所有数据报文没有丢失的情况下，拥塞窗口增大一个MSS数值，将原本在慢启动的指数增长，变为线性增长，减缓增长速率。

■ 快速恢复



■ 当发生超时重传时，则代表当前网络拥塞严重，此时拥塞窗口更新为一个MSS，慢启动门限设定为拥塞时窗口大小的一半，启动慢启动算法增加。

■ 当发生快速重传时，则代表当前网络拥塞一般，此时慢启动门限设定为拥塞时窗口大小的一半，而拥塞窗口设定为新的慢启动门限值，采用拥塞避免算法。

TCP面向连接

TCP连接建立 --- 三次握手

目的：

1. 允许双方进行参数协商
2. 对设备进行资源分配
3. 告知对端自己的存在，已经告知对端做好数据传输准备

C/S架构 --- 客户端/服务端架构 --- 谁先发送数据，谁为客户端。

规则：SYN被置位的TCP报文段，不能携带数据；但是需要消耗1个序列号。

规则：普通的TCP确认报文，可以携带数据；但是如果不携带数据，则不消耗序列号，即发送的下一个数据段的序列号仍为上一个报文的序列号。

规则：收到普通的ACK报文，本地不会使用ACK确认报文进行回复。

问题1：初始序列号(ISN)是怎么产生的，是随机值还是固定值？ --- 随机值

初始序列号 --- 通过源目地址、源目端口和随机因子，通过MD5算法计算得出。

- 1、安全
- 2、数据错乱

问题2：TCP序号绕回问题

序列号 -- 32bit组成

带来的问题：

- 1、重复序号，导致接收方错误的将重复报文认为是新收到的报文
- 2、延迟确认

解决：通过时间戳来设定序号的有效时间。

问题3：什么是时间戳？

```
1 Kind: 8 // 标记唯一的选项类型，比如window scale是3
2 Length: 10 bytes // 标记Timestamps选项的字节数
3 ++++++
4 | Kind=8 | Length=10 | TS Value (TSval) | TS Echo Reply (TSecr) |
5 ++++++
6 | 1 | 1 | 4 | 4
```

注意：时间戳是一个双向选项；如果一方不开启，则双方都不能使用；该参数是否开启，决定于TCP三次握手的前两次SYN报文中。

如果本地收到的报文中的TSecr参数不等于之前接收到的TSval参数，则本地发送RST报文，重新建立连接。

问题4：沾包问题，沾指的是应用层数据；发送方可以通过一个TCP报文段发送多个应用层数据包，导致接收方收到后，必须区分那些字节属于哪一个应用层数据。

解决方案：

1. 对于定长报文，每次应用程序读取固定长度即可。
2. 对于变长报文
 - 在包头的位置，多携带一个参数，该参数用来标识本数据的总长度。
 - 通过在不同报文之间增加分隔符实现 --- 例如FTP会增加\r\n字符

问题5：如果TCP连接已经建立成功，客户端突然故障？

解决思路：每一条TCP连接存在一个保活计时器，一般为2小时；一端会发送TCP探测报文段，每隔75秒发送一次，如果连续10次没有得到对端的回复，则认为对端出现故障，关闭连接。

问题6：一个TCP连接中，存在两个客户端时，能否建立连接？

TCP连接断开 --- 四次挥手

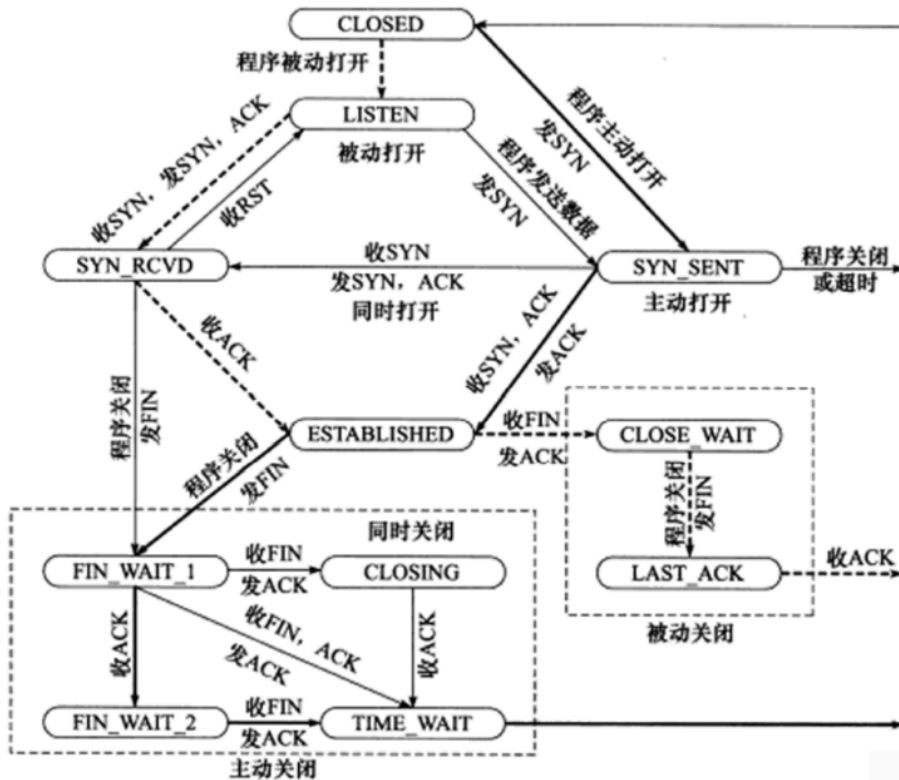
规则：FIN被置位的TCP报文段，不能携带数据；但是需要消耗1个序列号。

问题1：四次挥手过程，能否简化为三次挥手？可以，前提是服务端数据也发送完毕。

问题2：Time-Wait状态，为什么要等待2*MSL时间。

MSL --- 报文最大生存时间 --- 代表某个报文在网络中存在的最长时间，单位为秒
一般为2分钟。

问题3：如果两个端点，同时发送FIN报文，断开连接，是否可以正常断开？



第二部分，园区网

万物互联 --- 数据之间的互联计算
计算、级联、感知

